

CMS & Commerce Integration Plan

Vercel · WordPress · WooCommerce · Firebase Storage

UPDATED SEPTEMBER 10, 2026

Host the customer-facing React/Vite/Mantine app on Vercel. Administrators work exclusively in WordPress/WooCommerce. Use Firebase Storage for public media and protected digital files.

1. Define what each platform owns

Platform	Responsibility
Vercel	Host the React/Vite storefront, deployments, and preview environments.
Vercel Functions	Run secure API integrations, webhook handlers, and download authorization.
WordPress	Provide the sole admin interface for pages, posts, services, assets, FAQs, site settings, and staff access on separate hosting.
WooCommerce	Own sellable products, prices, coupons, customer records, orders, and refunds; connect Stripe as the payment gateway.
Firebase Storage	Store public product media and private PDFs, ZIP files, and other deliverables.
Firestore (optional)	Store supporting records, such as form submissions and integration jobs, only where needed.

- Firebase Hosting is not required. Vercel Functions will handle the planned backend integrations; Firebase Cloud Functions are optional for later needs.
- Keep WooCommerce authoritative for prices and orders. Any asset or service listing sold through checkout must reference its WooCommerce product.
- Keep layouts and responsive behavior in React/Mantine. Staff edit structured content in WordPress and preview changes on the React site.

Primary flow: React on Vercel → WordPress/WooCommerce APIs. Privileged operations → Vercel Functions. Approved file access → Firebase Storage.

Planning basis: the current app, LONGLIFE DIGITAL LLC.pdf, and MVPs.pdf. The PDFs inform this proposal; their broader business recommendations do not automatically become implementation tasks.

Content & application migration

2. Organize and migrate the content

- Inventory hardcoded data in `src/constants/data.js`, page components, navigation, footer, and chatbot configuration. Review sample products, domains, testimonials, and statistics before publishing them.
- Define structured fields for headings, descriptions, images, buttons, featured sections, contact details, and page metadata. Expose approved content through the WordPress REST API.
- Services: descriptions, packages, starting prices, and consultation options. Products: catalog details, categories, WooCommerce prices, and restricted file references. Assets: domains and ready-to-launch websites, availability, asking prices, and inquiry options.
- Import the approved initial catalog with stable IDs and slugs. Link related service and asset entries to WooCommerce products where direct checkout applies.

3. Establish the backend connections

- Configure separate staging and production CMS, storage, and payment settings. Vercel preview deployments use staging resources and test payment credentials.
- Build shared API services and React hooks for customer features. Use WordPress APIs for content and WooCommerce Store API for carts. Keep privileged integration credentials on the server; staff administration stays in WordPress.
- Build on the existing root-level `api/` structure using Vercel Functions. Add webhook, integration, and protected-download endpoints; keep secrets in server environment variables.
- Add a WordPress upload/linking integration for Firebase files so staff manage media and product deliverables in one admin interface. Restrict uploads by user permission, file type, and size.

4. Connect the existing pages gradually

- Start with the homepage, navigation, footer, and general pages. Then connect the shop, product details, services, assets, and blog.
- Add loading indicators, empty states, error handling, caching, and refresh behavior after publication. Keep unpublished content behind authenticated preview access.
- Introduce stable page and product URLs for bookmarks, shared links, account access, and checkout returns. Plan page metadata and prerendering where needed for search indexing.
- Preserve the responsive presentation and customer interactions while replacing their data sources. Remove hardcoded fallbacks after each migrated feature is verified.

Checkout & digital delivery

5. Move commerce into WooCommerce

- First prove one complete purchase using the selected WooCommerce gateway. Validate the headless payment integration early, including required payment data, redirects, and customer authentication challenges.
- Keep checkout inside React where the gateway supports the required experience. Evaluate a WooCommerce-hosted checkout as an alternative before implementation; a fully headless gateway flow is not assumed to work automatically.
- Send product IDs and quantities to WooCommerce and use its calculated totals, discounts, availability, and applicable taxes. Preserve cart identity across navigation and checkout.
- Have the gateway confirm payment to WooCommerce. Verify WooCommerce webhook signatures in Vercel Functions and confirm paid order status before fulfillment. Process retries without duplicating delivery.
- Replace the current standalone Stripe payment flow once the WooCommerce replacement is verified. Display persisted order numbers and status from WooCommerce.

6. Implement secure fulfillment

- Separate public product previews from private purchased files in Firebase Storage. Keep private storage references in restricted backend metadata, outside public product responses.
- On each download request, a Vercel Function verifies customer access and purchase eligibility, then issues a short-lived signed URL. Files transfer directly from storage. Signed links can be used by anyone holding them until expiry.
- Use the right fulfillment process: automatic downloads for digital products, onboarding for services, and a tracked ownership-transfer process for domains and websites. Prevent a unique asset from being sold twice.
- Send purchase confirmations and download-access instructions. Provide account-based access to previous purchases and a verified retrieval flow if guest checkout is enabled.
- Handle failed payments, refunds, and canceled orders consistently. Stop issuing new download links when access is revoked; already issued signed links remain usable until they expire.

First milestone: one CMS-driven product → successful test payment → persisted WooCommerce order → verified access → protected Firebase download.

Accounts, rollout & references

7. Centralize administration in WordPress

- Keep customer sign-up/sign-in, purchase history, and downloads in React. Use WordPress/WooCommerce as the primary identity and define how Vercel verifies customer sessions. Firebase does not require a second customer account for this design.
- Use WordPress/WooCommerce as the only admin interface. Assign Editor, Shop Manager, or Administrator roles according to responsibilities, with permissions enforced server-side.
- Migrate product/contact editing, team access, orders, and subscriber management into WordPress screens or integrations. Staff manage Firebase files there; no React management screens are retained.
- After workflow validation, remove the React admin dashboard, admin login and shortcut, edit/delete controls, and browser-only role/password code. Preserve customer account and purchase screens.
- Persist form submissions and newsletter subscriptions, with staff management in WordPress. Confirm their storage owner and email service. Supply the chatbot with current published content.

8. Roll out in manageable stages

- Foundation: confirm WordPress hosting, staff roles, environments, gateway support, customer sessions, catalog, and storage access. Prove the single-product purchase and delivery milestone.
- Migration: connect the remaining pages, catalog, customer accounts, and forms. Verify WordPress publishing, preview, file uploads, and all staff workflows before retiring the React admin screens.
- Launch: verify payments, refunds, webhook retries, customer access, staff permissions, content updates, and mobile layouts. Confirm admin controls are removed from React. Add logs, retries, and backups.
- Keep a rollback path and staging data isolated from production. Retire obsolete hardcoded records and payment endpoints after validation. Optional Firestore or later business features receive their own scope.

Official references

Vercel: Vite and API functions

Firebase: Cloud Storage for web apps

WordPress: REST support for custom content

Google Cloud: signed file URLs

WooCommerce: Stripe integration

WordPress: staff roles and capabilities

WooCommerce: checkout and payment data

WooCommerce: Shop Manager permissions

WooCommerce: webhook integration

Source documents: LOGLIFE DIGITAL LLC.pdf and MVPs.pdf, provided by the user.